

Throughout the assignment, unless otherwise specified, Turing machines:

1. work with a single tape,
2. take input n as $n + 1$ contiguous 1s with the reading head starting at the leftmost nonempty cell given state q_0 ,
3. and the state symbols we can use are q_i , where i is a natural number.

Q1. Consider the following Turing machine:

```

q0 1 q1 b L
q1 b q2 1 R
q2 b q3 b R
q3 1 q3 1 R
q3 b q4 b R
q4 1 q4 1 R
q4 b q5 1 L
q5 1 q5 1 L
q5 b q6 b L
q6 1 q6 1 L
q6 b q0 b R
q0 b q7 b R

```

- (a) What function does this machine compute? Assume the output is the number of 1s left on the tape.
- (b) Describe the tape, state, and head position after halting on input n .
- (c) Describe the numbers of steps and cells used as a function in input size, then give asymptotic descriptions.

Solution

(a) **Function computed**

The input number n is encoded as $n + 1$ contiguous 1's on the tape, with the head starting at the leftmost 1 in state q_0 .

Let $k = n + 1$ denote the number of input 1's. Each iteration of the machine removes one 1 from the left block and appends one 1 to the far right of the tape. This process repeats until all original 1's have been processed. After k iterations, the machine halts.

At that point the tape contains $2k$ total 1's. Therefore the machine computes the function

$$f(n) = 2(n + 1) = 2n + 2.$$

(b) **Configuration upon halting**

Let $k = n + 1$. When the machine halts:

- The tape contains

$$1^k b b 1^k,$$

i.e., k consecutive 1's, followed by two blank symbols, followed by another k consecutive 1's.

- The machine is in state q_7 .
- The head is positioned on the first 1 of the rightmost block (cell $k + 1$).

The total number of 1's left on the tape is $2k = 2(n + 1)$.

(c) **Time and space complexity**

Let $k = n + 1$ be the input size in unary.

Step count. From direct observation of executions:

Input	Steps
1	8
11	19
111	34
1111	53
11111	76

These values are exactly described by the quadratic function

$$T(k) = 2k^2 + 5k + 1.$$

Substituting $k = n + 1$, the number of steps as a function of n is

$$T(n) = 2(n + 1)^2 + 5(n + 1) + 1 = 2n^2 + 9n + 8.$$

Space usage. The leftmost nonblank cell used is position -1 , and the rightmost nonblank cell is position $2k$. Hence the number of tape cells used is

$$S(n) = (2k - (-1)) + 1 = 2k + 2 = 2n + 4.$$

Asymptotic bounds.

$$\begin{array}{l} \text{Time complexity: } \Theta(n^2) \\ \text{Space complexity: } \Theta(n) \end{array}$$

Q2. Let A and B be decidable subsets of $\mathbb{N}$, and assume that M_A and M_B are Turing machines which compute the corresponding characteristic functions.

These machines give an empty tape if the input is not in the set, and a tape with exactly a single 1 on it if the input is in the set.

Assume these machines use only the cells used by the input (i.e., they do not need more space than the one already occupied by the input). Suppose these two machines always halt in state q_{16} , and if the output tape is not empty, the head ends up positioned on the nonempty cell.

Let M denote the Turing machine in Question 1 above.

Consider the following Turing machine:

```

M
M̃_A
q23 1 q24 b L
q23 b q25 b L
q25 b q25 b L
q25 1 q26 b L
q26 1 q26 b L
q24 b q24 b L
q24 1 q27 1 L
q27 1 q27 1 L
q27 b q28 b R
M̃_B

```

where $\tilde{M}_A$ is the machine we obtain from M_A by replacing every q_i with q_{i+7} , and $\tilde{M}_B$ is the machine we obtain from M_B by replacing every q_i with q_{i+28} .

This Turing machine computes the characteristic function of some set, find that set.

Solution

Let the input number be encoded by $k = n + 1$ contiguous 1's as before. Recall from Question 1 that running M on an input of k ones yields the tape

$$1^k \ b \ b \ 1^k,$$

and M halts in state q_7 with the head on the leftmost 1 of the right block. (We will call the left block the *left copy* and the right block the *right copy*.)

Now consider the composed machine in the statement. Because every state of M_A was renamed by adding 7, the start state of $\tilde{M}_A$ is q_7 and so $\tilde{M}_A$ begins immediately after M

halts, with the head at the leftmost 1 of the right copy. By hypothesis $\tilde{M}_A$ (like M_A) uses only the cells of its input region (here: the right copy), and when it halts it is in state q_{23} . Two possibilities occur.

Case 1: $n \in A$.

Then $\tilde{M}_A$ leaves exactly one 1 in the right copy and halts in state q_{23} with the head on that 1. The transition

$$q_{23} \ 1 \ q_{24} \ b \ L$$

erases that single 1 and moves left into the two-blank separator. The rules for q_{24} move the head left over the separator blanks until the head meets the rightmost 1 of the left copy; the machine then moves left across the left copy (using q_{27}) until it reaches the blank immediately to the left of the left copy and executes

$$q_{27} \ b \ q_{28} \ b \ R,$$

which moves the head right once and enters state q_{28} .

But q_{28} is exactly the start state of $\tilde{M}_B$ (since every state of M_B was offset by +28). Thus, in this branch $\tilde{M}_B$ is started with its head at the leftmost 1 of the left copy (i.e., the left copy is presented as the input to $\tilde{M}_B$). After $\tilde{M}_B$ runs, it outputs a single 1 on the left copy iff $n \in B$; otherwise the left copy is left empty. In either subcase the final tape after $\tilde{M}_B$ halts is

$$\begin{cases} 1 & \text{(a single 1 somewhere in the left region), if } n \in B, \\ \text{empty tape (all blanks)} & \text{if } n \notin B. \end{cases}$$

Hence when $n \in A$ the composed machine outputs a single 1 $\iff n \in B$. So in this branch the composed machine accepts exactly those $n \in A \cap B$.

Case 2: $n \notin A$.

Then $\tilde{M}_A$ leaves the right copy empty and halts in state q_{23} with the head reading a blank. The transition

$$q_{23} \ b \ q_{25} \ b \ L$$

moves left into the separator; the q_{25} rules move left over separator blanks until the head reaches the rightmost 1 of the left copy, whereupon

$$q_{25} \ 1 \ q_{26} \ b \ L$$

and the q_{26} rule

$$q_{26} \ 1 \ q_{26} \ b \ L$$

erase 1's of the left copy while moving left. Thus the entire left copy is erased. (When the left copy has been entirely erased the machine has no further transition from q_{26} on a blank and therefore halts with an empty tape.) So in this branch the composed machine halts with an empty tape (i.e., it *rejects*).

Conclusion:

Combining both cases: the composed machine leaves exactly one 1 on the tape precisely when (i) $\tilde{M}_A$ accepts and then (ii) $\tilde{M}_B$ accepts the left copy. Equivalently, the composed machine accepts exactly those n that lie in both A and B .

The composed machine computes the characteristic function of $A \cap B$.

Q3. Recall that a polynomial in $\mathbb{Z}$ with one variable x is an expression of the form $\sum_{i=0}^m a_i x^i$ where, for all i , $a_i \in \mathbb{Z}$. The a_i 's are known as the *coefficients*.

The same concept can be extended to include more than one variable. A polynomial in $\mathbb{Z}$ with k variables $x_1, \dots, x_k$ is an expression of the form

$$\sum_{i_1, i_2, \dots, i_k \in \{0, \dots, m\}} a_{i_1, i_2, \dots, i_k} x_1^{i_1} \dots x_k^{i_k}$$

where all the coefficients $a_{i_1, i_2, \dots, i_k}$ are integers.

If the above description is confusing to you, solve the following exercise and do not submit it (just for you to see an example).

Exercise: Consider the polynomial $x_1^2 - 2y_1^3 + y_1y_2^3$. Complete the following:
 $a_{2,0,0,0} = 1$, $a_{0,0,3,0} = -2$, $a_{0,0,1,3} = ??$, $a_{0,0,0,0} = ??$, $a_{1,1,1,1} = ??$.

Let j be a positive natural number, and let A be a subset of $\mathbb{N}^j$. We say that A is a *Diophantine subset* of $\mathbb{N}$ if there exists a polynomial equation in $\mathbb{Z}$ with at least j variables, say, $P(x_1, \dots, x_j, y_1, \dots, y_k) = 0$ (it is possible that there might be no y_i 's) such that

For all $x_1, \dots, x_j \in \mathbb{N}$,

$$(x_1, \dots, x_j) \in A \iff (\exists y_1, \dots, y_k \in \mathbb{N}) [P(x_1, \dots, x_j, y_1, \dots, y_k) = 0].$$

- (a) Prove that the set of even numbers is a Diophantine subset of $\mathbb{N}$.
- (b) Prove that the relation $<$ is a Diophantine subset of $\mathbb{N}^2$.
- (c) Prove that every Diophantine subset of $\mathbb{N}$ is Turing recognizable (i.e., computably enumerable).

Solution

- (a) **The set of even numbers is Diophantine.**

Let

$$A = \{x \in \mathbb{N} \mid x \text{ is even}\}.$$

An integer x is even if and only if it can be written as $x = 2y$ for some $y \in \mathbb{N}$. This condition can be expressed using the polynomial

$$P(x, y) = x - 2y.$$

Then, for every $x \in \mathbb{N}$,

$$x \in A \iff (\exists y \in \mathbb{N}) P(x, y) = 0.$$

Thus membership in the set of even numbers is equivalent to the existence of a natural-number solution to a polynomial equation.

Therefore, the set of even numbers is a Diophantine subset of $\mathbb{N}$.

(b) **The relation $<$ is Diophantine.**

Let

$$R = \{(x, y) \in \mathbb{N}^2 \mid x < y\}.$$

The inequality $x < y$ holds if and only if the difference between y and x is at least 1. Equivalently, there exists $k \in \mathbb{N}$ such that

$$y = x + 1 + k.$$

This condition can be rewritten as the polynomial equation

$$Q(x, y, k) = x + 1 + k - y = 0.$$

Hence,

$$(x, y) \in R \iff (\exists k \in \mathbb{N}) Q(x, y, k) = 0.$$

Therefore, the relation $<$ is a Diophantine subset of $\mathbb{N}^2$.

(c) **Every Diophantine subset of $\mathbb{N}$ is Turing recognizable.**

Let $A \subseteq \mathbb{N}$ be a Diophantine set. By definition, there exists a polynomial

$$P(x, y_1, \dots, y_k) \in \mathbb{Z}[x, y_1, \dots, y_k]$$

such that for all $x \in \mathbb{N}$,

$$x \in A \iff (\exists y_1, \dots, y_k \in \mathbb{N}) P(x, y_1, \dots, y_k) = 0.$$

Recognition procedure: We describe a Turing machine that recognizes A . Given an input x , the machine systematically enumerates all tuples $(y_1, \dots, y_k) \in \mathbb{N}^k$. For each tuple, it computes the value of the polynomial $P(x, y_1, \dots, y_k)$, which is a finite and computable operation.

- If $x \in A$, then by definition there exists at least one tuple $(y_1, \dots, y_k)$ such that $P(x, y_1, \dots, y_k) = 0$. Since all tuples are eventually checked, the machine will eventually find such a tuple and halt, accepting x .
- If $x \notin A$, no such tuple exists, and the machine will never halt.

Thus, membership in A can be recognized by a Turing machine.

Therefore, every Diophantine subset of $\mathbb{N}$ is Turing recognizable (computably enumerable).

Q4. Let $f: \mathbb{N} \rightarrow \mathbb{N}$ be a computable function. Suppose that f is bijective. Prove that the inverse function of f is also computable.

Solution

Proof. Let $f: \mathbb{N} \rightarrow \mathbb{N}$ be a computable bijection. We will show that f^{-1} is computable.

Define the binary predicate

$$R(x, y) : \iff f(x) = y.$$

Since f is computable, given x and y we can compute $f(x)$ and check whether $f(x) = y$. Hence $R(x, y)$ is a decidable (computable) predicate. Additionally, because f is bijective, for every $y \in \mathbb{N}$ there exists a *unique* $x \in \mathbb{N}$ such that $R(x, y)$ holds.

We now define

$$g(y) = \mu x R(x, y),$$

where $\mu x R(x, y)$ denotes the *least* natural number x for which $R(x, y)$ is true.

This definition means the following: given an input y , we systematically test $x = 0, 1, 2, \dots$ and return the first x such that $f(x) = y$. Since f is onto, such an x is guaranteed to exist, and since f is one-to-one, this x is unique. Thus the search always halts and returns exactly $f^{-1}(y)$.

Because R is decidable and for each y some x satisfying $R(x, y)$ exists, the (unbounded) minimization operator μ yields a total computable function g . By construction, $g(y)$ is the unique x such that $f(x) = y$, hence $g = f^{-1}$.

Algorithmic description: On input y , enumerate $x = 0, 1, 2, \dots$, compute $f(x)$ for each x , and output the first x such that $f(x) = y$. This procedure halts for every y and computes $f^{-1}(y)$.

Therefore, f^{-1} is computable.

□

Q5. Let $f: \mathbb{N} \rightarrow \mathbb{N}$ be a computable function. Prove that, for every $x \in \mathbb{N}$, the set $f^{-1}(x)$ is decidable.

Solution

Proof. Fix $x \in \mathbb{N}$ and consider the set

$$A_x := f^{-1}(x) = \{y \in \mathbb{N} \mid f(y) = x\}.$$

To show that A_x is decidable, it suffices to give a total computable procedure that, on input y , decides whether $y \in A_x$.

Since f is computable, there exists an algorithm that, given any input y , computes the value $f(y)$ and always halts. Using this algorithm, we define the following decision procedure for membership in A_x :

Decision Procedure:

- (a) On input y , compute $f(y)$.
- (b) If $f(y) = x$, accept (output “yes”); otherwise reject (output “no”).

Intuition: To determine whether y belongs to $f^{-1}(x)$, we simply evaluate the function f at y and check whether the result equals x . No search or minimization is required. Because f is computable, this process always halts and yields a definite answer.

The procedure always halts since computing $f(y)$ halts for every y , and equality of natural numbers is decidable. Therefore, the characteristic function

$$\chi_{A_x}(y) = \begin{cases} 1 & \text{if } f(y) = x, \\ 0 & \text{otherwise} \end{cases}$$

is total and computable.

Therefore, $A_x = f^{-1}(x)$ is decidable.

□

Q6. Let A be a set of natural numbers that happens to be the range of some computable function. Prove that there exists a Turing machine which halts exactly on the elements of A .

Solution

Proof. Suppose $A \subseteq \mathbb{N}$ is the range of a computable function

$$f: \mathbb{N} \rightarrow \mathbb{N}.$$

Thus

$$A = \{y \in \mathbb{N} \mid \exists x \in \mathbb{N} (f(x) = y)\}.$$

We construct a Turing machine M that, on input y , halts if and only if $y \in A$.

Since f is computable, there exists a Turing machine that, given any input x , computes $f(x)$ and halts. The machine M operates as follows on input y :

Machine M on input y :

- (a) Enumerate the natural numbers $x = 0, 1, 2, \dots$
- (b) For each x , compute $f(x)$.
- (c) If for some x we find that $f(x) = y$, then halt.

Correctness:

- If $y \in A$, then by definition there exists some x such that $f(x) = y$. When the enumeration reaches this x , the machine detects the equality and halts.
- If $y \notin A$, then for every x we have $f(x) \neq y$, and the machine continues the enumeration forever and never halts.

Therefore, M halts exactly on those inputs y that belong to A .

□

Q7. Let A be the set of values on which a Turing machine M halts. Assume that A is not empty. Prove that A is the range of some computable function.

Solution

Proof. Let M be a Turing machine and define

$$A = \{y \in \mathbb{N} \mid M \text{ halts on input } y\}.$$

Assume that $A \neq \emptyset$. We will construct a total computable function whose range is exactly A .

Idea: Although membership in A is not decidable, the predicate “ M halts on input s within t steps” is decidable for fixed s and t , since it can be checked by simulating M for exactly t steps. We use this observation to enumerate all inputs on which M halts.

Fix a computable enumeration $(p_0, p_1, p_2, \dots)$ of $\mathbb{N}^2$, where $p_i = (s_i, t_i)$. Consider the following enumeration procedure.

Enumeration Procedure:

- For $i = 0, 1, 2, \dots$, simulate M on input s_i for exactly t_i steps.
- If M halts within those t_i steps, output the value s_i .

If $s \in A$, then M halts on input s after some finite number of steps t_s . Hence for every $t \geq t_s$, the pair (s, t) appears in the enumeration and causes s to be output. Therefore, every element of A appears (in fact infinitely often) in the resulting sequence of outputs. Since A is nonempty, the enumeration produces infinitely many outputs.

Defining the function f : Using the enumeration procedure above, we define a function

$$f : \mathbb{N} \rightarrow \mathbb{N}$$

as follows. On input n , we simulate the enumeration procedure step by step, keeping a counter of how many values have been output so far. Once the enumeration has produced exactly $n + 1$ outputs, we halt and return the last value produced.

Basically, think of the enumeration as producing an infinite list

$$e_0, e_1, e_2, \dots$$

of natural numbers (with repetitions allowed), where each e_i is an input on which M halts. The function $f(n)$ simply returns the n -th element of this list. Because the enumeration produces infinitely many outputs, the n -th output always exists, so f is total.

The function f is computable because the enumeration procedure is computable and the process of counting outputs and stopping at the $(n + 1)$ -st output is a finite, effective computation. Moreover, f is total since, as shown above, the enumeration produces infinitely many outputs.

Verification that $\text{range}(f) = A$:

- Every value output by the enumeration is some s such that M halts on input s . Therefore every value of f belongs to A , and hence $\text{range}(f) \subseteq A$.
- Conversely, if $s \in A$, then M halts on input s after some finite number of steps. As shown earlier, this implies that s appears (in fact infinitely often) among the outputs of the enumeration. Hence s occurs as $f(n)$ for some n , and thus $s \in \text{range}(f)$.

Thus $\text{range}(f) = A$.

□

Q8. Let A and B be two decidable sets. Prove that $A \cap B$ and $A \cup B$ are also decidable sets.

Solution

Proof. Since A and B are decidable, there exist total computable characteristic functions (or decision procedures)

$$\chi_A, \chi_B : \mathbb{N} \rightarrow \{0, 1\},$$

where $\chi_A(x) = 1$ iff $x \in A$ and $\chi_B(x) = 1$ iff $x \in B$.

Define functions for the intersection and union by

$$\chi_{A \cap B}(x) := \chi_A(x) \cdot \chi_B(x), \quad \chi_{A \cup B}(x) := \max\{\chi_A(x), \chi_B(x)\}.$$

Both definitions use only computable operations (multiplication and maximum) on the computable functions χ_A and χ_B , so $\chi_{A \cap B}$ and $\chi_{A \cup B}$ are total computable functions from $\mathbb{N}$ to $\{0, 1\}$. Hence $A \cap B$ and $A \cup B$ are decidable.

Additionally, we can also do/see that on input x :

To decide membership in $A \cap B$:

- Run the decider for A on x and the decider for B on x (sequentially or in parallel).
- Accept iff both deciders accept; otherwise reject.

To decide membership in $A \cup B$:

- Run the decider for A on x and the decider for B on x .
- Accept iff at least one accepts; otherwise reject.

These procedures always halt because the deciders for A and B always halt.

Thus $A \cap B$ and $A \cup B$ are decidable.

□

Q9. Let A and B be two semi-decidable (i.e., computably enumerable) sets. Prove that $A \cap B$ and $A \cup B$ are also semi-decidable sets.

Solution

Proof. Recall that a set is *semi-decidable* if there exists a Turing machine that halts exactly on the elements of the set.

Since A and B are semi-decidable, there exist Turing machines M_A and M_B such that

$$M_A \text{ halts on input } x \iff x \in A, \quad M_B \text{ halts on input } x \iff x \in B.$$

Definition (Dovetailing). To simulate two possibly non-halting Turing machines, we interleave their computations by simulating one step of the first machine, then one step of the second, and repeating. This way, if either machine halts, it is detected.

Union: $A \cup B$ is semi-decidable.

We construct a Turing machine $M_{\cup}$ that halts on input x if and only if $x \in A \cup B$.

Machine $M_{\cup}$ on input x :

- (a) Simulate $M_A(x)$ and $M_B(x)$ using dovetailing (alternating one step of each).
- (b) If either simulation halts, then halt.

Correctness:

- If $x \in A \cup B$, then $x \in A$ or $x \in B$, so at least one of $M_A(x)$ or $M_B(x)$ halts. By dovetailing, $M_{\cup}$ eventually detects this and halts.
- If $x \notin A \cup B$, then neither $M_A(x)$ nor $M_B(x)$ halts, so $M_{\cup}$ runs forever.

Thus $A \cup B$ is semi-decidable.

Intersection: $A \cap B$ is semi-decidable.

We now construct a Turing machine $M_{\cap}$ that halts on input x if and only if $x \in A \cap B$.

Machine $M_{\cap}$ on input x :

- (a) Simulate $M_A(x)$ and $M_B(x)$ using dovetailing.
- (b) Halt only after *both* simulations have halted.

Correctness:

- If $x \in A \cap B$, then $x \in A$ and $x \in B$, so both $M_A(x)$ and $M_B(x)$ halt. Dovetailing ensures that both halting events are eventually observed, and thus $M_{\cap}$ halts.

- If $x \notin A \cap B$, then at least one of the machines does not halt, so $M_{\cap}$ never halts.

Therefore $A \cap B$ is semi-decidable.

The class of semi-decidable sets is closed under union and intersection.

□